PROJECT SYNOPSIS
On

"LexAI – AI-Powered Legal Assistant & Contract Intelligence Platform"

Submitted to

Chhatrapati Shivaji Maharaj University, Panvel, Navi Mumbai

for the partial fulfillment of requirement for the Degree of

MASTER OF COMPUTER APPLICATION

Submitted By

Rohan Bongane
Harsh Jha
Aishwariya Bhunia
Nitu Sharma

Guide

Dr. Surekha Kohle

Department of Computer Science & Information Technology
Chhatrapati Shivaji Maharaj University, Panvel, Navi Mumbai

Academic Year:
2026–27

1. Introduction

The legal domain is characterized by densely packed terminology and exceptionally lengthy documents, creating significant barriers to entry for individuals and small businesses. Commercial contracts, non-disclosure agreements, and terms of service are often obfuscated by specialized jargon. Manually reviewing these documents requires immense cognitive effort and frequently necessitates consulting expensive legal professionals to comprehend basic rights, obligations, and hidden risks.

LexAI is an AI-assisted legal information and document intelligence platform designed to bridge this accessibility gap. Utilizing a Flask-based web architecture, LexAI allows users to upload complex legal contracts in PDF or DOCX formats. The platform automates the heavy lifting of document review by extracting, chunking, and processing the text to perform deep semantic analysis.

To mitigate the critical issue of "hallucination" commonly associated with Large Language Models (LLMs), LexAI implements a robust Retrieval-Augmented Generation (RAG) architecture powered by the Google Gemini LLM. By grounding the AI's responses exclusively in the retrieved text of the user's specific document or an admin-curated Legal Knowledge Base, LexAI ensures that generated answers are contextually relevant, highly accurate, and factually verifiable. It is explicitly noted that LexAI acts as an informational assistant and does not replace professional, jurisdictional legal counsel.

2. Literature Review

2.1 Legal Document Summarization

Research by Jain et al. [1] highlights the complexities of generating accurate summaries for legal documents, emphasizing the necessity of preserving factual integrity while condensing extensive case texts. Traditional extractive methods often miss semantic nuance, while generative models risk fabricating legal precedent. This underlines the need for deterministic chunking strategies prior to abstraction, which LexAI adopts to generate safe executive summaries.

2.2 Domain-Specific NLP Models

The uniqueness of legal vocabulary requires specialized NLP approaches. Chalkidis et al. [2] demonstrated with LEGAL-BERT that generic language models frequently misinterpret legal nuances, necessitating domain-aware processing. While LexAI utilizes the all-MiniLM-L6-v2 model for generalized embedding efficiency, it relies on structured JSON-schema prompting with the Gemini LLM to enforce domain-specific analytical constraints during generation.

2.3 Retrieval-Augmented Generation (RAG)

The integration of non-parametric memory with parametric LLMs was formalized by Lewis et al. [3], establishing RAG as the standard for knowledge-intensive NLP tasks. Their findings proved that querying an external vector index drastically reduces factual hallucinations. LexAI incorporates this exact architecture by utilizing ChromaDB as a semantic vector store, ensuring the AI only processes text explicitly present in the uploaded contract.

2.4 Challenges in Legal NLP and Question Answering

Recent surveys by Ariai et al. [4] and Martinez-Gil [5] underscore the ongoing challenges in legal QA systems, particularly concerning model explainability, privacy, and user trust. Public generative AI interfaces pose significant security risks for confidential contracts. Furthermore, unconstrained models often hallucinate standard practices instead of citing specific contractual terms.

2.5 Research Gap

While existing literature proves the efficacy of RAG and domain-specific LLMs, there remains a gap in accessible, locally verifiable systems that combine private document analysis, conversational querying, and a curated general legal knowledge base within a single, securely isolated platform. LexAI addresses this gap by enforcing strict database-level filtering (document ID isolation), ensuring users only receive answers grounded in their private contracts or the admin-curated repository, thereby maximizing privacy and mitigating hallucinations.

3. Aims & Objective

The primary aim of the LexAI project is to develop a secure, highly accurate, and accessible web-based platform that utilizes Artificial Intelligence to demystify complex legal documents.

Specific Objectives:

- To securely authenticate users and isolate their uploaded legal documents.
- To extract and mathematically embed text from uploaded PDF and DOCX files.
- To implement a Retrieval-Augmented Generation (RAG) pipeline utilizing ChromaDB for highly accurate semantic search.
- To automatically generate structured executive summaries highlighting key obligations and risks using the Gemini API.
- To provide a conversational document chatbot strictly grounded in the user's specific uploaded contract.
- To maintain a deterministic fallback mechanism capable of serving general legal knowledge even if external AI APIs become unavailable.

4. Problem Definition

The sheer volume and archaic terminology of legal text make contracts largely inaccessible to the general public. When presented with standard legal agreements, individuals face several challenges:

- **Manual Review Effort:** Reading and comprehending dense, multi-page documents requires prohibitive amounts of time.
- **Hidden Risks:** Critical obligations, termination conditions, and liability waivers are frequently obfuscated by legalese, making manual identification error-prone.
- **Retrieval Friction:** Locating the exact clause for a specific question (e.g., "What is the governing law?") via basic keyword search often fails if exact terminology is not used.
- **Unreliable Public AI:** Pasting confidential contracts into public generative AI chatbots poses severe privacy risks, and generic LLMs frequently hallucinate legal answers rather than citing the provided text.

5. Proposed Model

LexAI proposes a decoupled client-server architecture to provide a secure environment for document intelligence.

Architecture Components:

- **Frontend:** Responsive web interface built with HTML5, Bootstrap 5, and JavaScript. Uses DOMPurify to sanitize AI responses and MathJax to render complex logic.
- **Backend Application:** A robust Python Flask server managing session authentication (Flask-Login, Bcrypt), secure routing, and orchestration of the AI pipeline.
- **Relational Database:** MySQL (via SQLAlchemy) stores user credentials, document metadata, and conversation histories.
- **Vector Database & Embeddings:** Extracted document text is processed by a local SentenceTransformers model (all-MiniLM-L6-v2) to generate 384-dimensional embeddings, which are stored in ChromaDB for high-speed semantic retrieval.
- **Generative LLM:** The Google Gemini API processes the retrieved context alongside strict system prompts to generate factual, grounded responses.

6. Methodology

The operational methodology of LexAI is structured into a sequential processing pipeline:

- **Secure Ingestion:** Users upload PDF or DOCX files. The backend validates file integrity and extracts raw unicode text using PyMuPDF and python-docx.
- **Contextual Chunking:** To accommodate LLM token limits, the text is sliced into manageable overlapping segments (e.g., 500 words with a 50-word overlap) to preserve contextual boundaries.
- **Vectorization:** Each chunk is converted into a mathematical vector embedding locally and indexed in ChromaDB, tagged strictly with the user's ID to prevent cross-contamination.

- **Semantic Retrieval (RAG):** When a user queries a document, their question is embedded into a vector. ChromaDB calculates mathematical similarities and retrieves the top matching chunks.
- **Grounded Generation:** The retrieved chunks are appended to a strict prompt instructing the Gemini API to answer based solely on the provided text. The generated response is formatted in Markdown and returned to the user interface.
- **Deterministic Fallback:** If the Gemini API experiences an outage, a Python fallback parser directly extracts and formats the retrieved raw database chunks, ensuring uninterrupted access to the Knowledge Base.

7. Plan of Work

The project execution was organized into distinct phases separating implemented features from future scope:

Implemented Phases:

- **Phase 1: Project Setup & Authentication.** Configuration of Flask, MySQL, and implementation of secure Bcrypt user registration and login.
- **Phase 2: Document Processing.** Implementation of secure file uploads, validation, and PyMuPDF/python-docx text extraction pipelines.
- **Phase 3: AI Legal Analysis.** Integration of the Gemini API with structured JSON prompts to generate automated executive summaries.
- **Phase 4: RAG & Chat Interfaces.** Deployment of local SentenceTransformers, ChromaDB vector indexing, and asynchronous Document Chat implementation.
- **Phase 5: Knowledge Base & Testing.** Development of the admin-curated legal Knowledge Base, fallback mechanisms, and comprehensive Pytest validation suites.

Future Scope (Planned):

- **Phase 6: Optical Character Recognition (OCR).** Integration of Tesseract OCR to process scanned, image-based legal documents.

- **Phase 7: Multilingual Support.** Expanding semantic models to analyze and translate legal concepts into regional languages.

8. Conclusion

The LexAI platform successfully demonstrates the practical application of Retrieval-Augmented Generation (RAG) in solving the accessibility crisis surrounding legal document comprehension. By integrating an intuitive Flask-based web interface with a robust, locally embedded vector pipeline, LexAI provides a secure, isolated environment for intelligent contract analysis.

Through automated executive summaries, semantic clause identification, and conversational querying, the system significantly reduces the cognitive load required to navigate dense legalese. Enforcing strict database isolation and utilizing deterministic fallback mechanisms ensures that the platform remains reliable, private, and highly resistant to AI hallucinations, representing a substantial technological advancement in making legal information transparent and understandable.

9. References

[1] Jain, D., Borah, M. D., & Biswas, A. (2021). "Summarization of legal documents: Where are we now and the way forward." **Computer Science Review**, 40, 100388.

[2] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., & Androutsopoulos, I. (2020). "LEGAL-BERT: The Muppets straight out of Law School." **Findings of the Association for Computational Linguistics: EMNLP 2020**.

[3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Riedel, S. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." **Advances in Neural Information Processing Systems (NeurIPS)**.

[4] Ariai, F., Mackenzie, J., & Demartini, G. (2025). "Natural Language Processing for the Legal Domain: A Survey of Tasks, Datasets, Models, and Challenges." **ACM Computing Surveys**, 58(6).

[5] Martinez-Gil, J. (2023). "A survey on legal question–answering systems." **Computer Science Review**, 48, 100551.

Prof. Dr. Surekha Kohle	**Dr. Praveen Gupta**
Guide	H.O.D.
CS/IT Department	CS/IT Department